

CompPar: Hierarquia de Memória, Otimizações de Cache e Pthreads

Localidade Espacial, Temporal, Tiling (Blocagem) e Paralelismo em Memória Compartilhada

Faculdade de Computação e Informática – FCI

Table of contents

1	Introdução e Objetivos	1
2	Fundamentos da Hierarquia de Memória e Disposição em C	2
3	Item 1: Inicialização Padronizada das Matrizes	2
4	Item 2: Impacto da Localidade Espacial (Linha vs. Coluna)	2
4.1	Código de Referência para Medição	3
4.2	Experimento Obrigatório	3
5	Item 3: Multiplicação de Matrizes Sequencial e Técnica de Blocagem (<i>Tiling</i>)	3
5.1	O Problema de Acesso no Algoritmo Canônico	3
5.2	A Solução por Blocagem (<i>Tiling</i>)	3
5.3	Análise Comparativa e Profiling com Valgrind / Cachegrind	4
5.3.1	Execução com o Cachegrind	4
5.3.2	Exemplo Real de Saída do Cachegrind	4
5.3.3	Guia Didático: Onde Olhar e Como Interpretar o Relatório	5
6	Item 4: Adicionando Paralelismo com Pthreads	6
6.1	Objetivos da Implementação Paralela	6
6.2	Cuidados com Falsa Partilha (<i>False Sharing</i>)	6
7	Questões de Reflexão Teórico-Prática	6
8	Diretrizes para Elaboração do Relatório de Laboratório	7
8.1	Formato e Identificação	7
8.2	Seções Obrigatórias do Relatório	7
9	CrITÉrios de Avaliação	8

1 Introdução e Objetivos

O desempenho de aplicações de computação científica em arquiteturas contemporâneas raramente é limitado apenas pela capacidade de processamento aritmético da CPU (FLOPS). Na maioria dos cenários reais, o fator preponderante de gargalo é o **acesso à hierarquia de memória** (*Memory Wall*): a taxa de transferência e a latência entre os registradores do processador, as memórias cache (L1, L2, L3) e a memória principal (DRAM).

Neste laboratório prático, investigaremos de forma empírica e analítica:

1. A diferença crítica de desempenho entre padrões de acesso que favorecem a **localidade espacial** (varredura por linha / *Row-Major*) versus padrões que a destroem (varredura por coluna / *Column-Major*).
 2. O impacto da técnica de **blocagem** (*tiling*) na multiplicação de matrizes densas, transformando o padrão de acesso para manter os dados ativos nos níveis mais rápidos da memória cache.
 3. O efeito das otimizações automáticas do compilador GCC (`-O0` versus `-O3`).
 4. A instrumentação e medição de faltas de cache (*cache misses* em L1 e Last Level Cache) por meio do utilitário `valgrind --tool=cachegrind` ou do subsistema `perf`.
 5. A integração do **paralelismo no nível de thread (Pthreads)** com as técnicas de exploração de cache, avaliando escalabilidade, *Speedup* e o risco de falsa partilha (*False Sharing*).
-

2 Fundamentos da Hierarquia de Memória e Disposição em C

Em linguagem C, matrizes bidimensionais estáticas e matrizes alocadas de forma linear contígua seguem a ordem **Row-Major** (linha por linha). Isso significa que os elementos de uma mesma linha i , como $A[i][0], A[i][1], \dots, A[i][N-1]$, residem em endereços contíguos de memória.

Quando o processador acessa uma palavra de memória que não está no cache (*cache miss*), ele não busca apenas o dado pontual de 4 ou 8 bytes: uma **linha inteira de cache** (*cache line*, tipicamente de **64 bytes**) é transferida da DRAM para o cache L1d.

- Para inteiros de 4 bytes (`int`), uma única linha de cache acomoda **16 elementos contíguos**.
 - **Varredura por Linha:** O acesso a $A[i][j]$ carrega 16 inteiros. Os próximos 15 acessos ($A[i][j+1]$ até $A[i][j+15]$) resultarão em acertos de cache (*cache hits*), promovendo alta eficiência.
 - **Varredura por Coluna:** Ao acessar $A[0][j]$ e em seguida $A[1][j]$, o salto de memória é de $N \times 4$ bytes. Se N for grande, a linha recém-carregada é descartada antes de ser reaproveitada, resultando em faltas de cache sistemáticas a cada iteração.
-

3 Item 1: Inicialização Padronizada das Matrizes

Para que os resultados computados sejam auditáveis e determinísticos em todas as etapas, utilize a inicialização matemática padrão:

```
/* Inicialização determinística para matrizes quadradas N x N */
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        A[i * N + j] = (double)(i + j);
        B[i * N + j] = (double)(i * j);
    }
}
```

Observação de Engenharia de Software: É fortemente recomendado utilizar blocos contíguos de memória (`double* A = malloc(N * N * sizeof(double))`) indexados como $A[i * N + j]$, em detrimento de matrizes de ponteiros (`double**`), pois o `malloc` linha a linha espalha ponteiros pelo heap e corrompe a previsibilidade das linhas de cache.

4 Item 2: Impacto da Localidade Espacial (Linha vs. Coluna)

Nesta etapa, implemente um programa que inicializa uma matriz quadrada A de dimensão $N \times N$ e contabiliza a quantidade de elementos pares contidos nela.

Você deve desenvolver **dois executáveis distintos**:

1. `varredura_linha`: laço externo em i (linhas) e laço interno em j (colunas).
2. `varredura_coluna`: laço externo em j (colunas) e laço interno em i (linhas).

4.1 Código de Referência para Medição

```
/* Ordem de Linha (Row-Major) - Excelente Localidade Espacial */
long pares = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if ((int)A[i * N + j] % 2 == 0) {
            pares++;
        }
    }
}

/* Ordem de Coluna (Column-Major) - Pobre Localidade Espacial */
long pares = 0;
for (int j = 0; j < N; j++) {
    for (int i = 0; i < N; i++) {
        if ((int)A[i * N + j] % 2 == 0) {
            pares++;
        }
    }
}
```

4.2 Experimento Obrigatório

Execute ambos os programas variando a ordem da matriz: $N \in \{512, 1024, 2048, 4096, 8192\}$. Meça os tempos de execução com alta precisão (`clock_gettime`) e calcule a razão de desaceleração (*slowdown* $= T_{\text{coluna}}/T_{\text{linha}}$).

5 Item 3: Multiplicação de Matrizes Sequencial e Técnica de Blocagem (*Tiling*)

A multiplicação clássica de matrizes $C = A \times B$ possui complexidade algorítmica $O(N^3)$. Para matrizes densas $N \times N$, a formulação padrão realiza:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

5.1 O Problema de Acesso no Algoritmo Canônico

No laço mais interno sobre o índice k :

- $A[i][k]$ é lido sequencialmente na mesma linha i (excelente localidade espacial).
- $C[i][j]$ permanece fixo nos registradores ou na mesma posição (excelente localidade temporal).
- **Porém**, $B[k][j]$ é acessado saltando linhas a cada passo ($B[0][j], B[1][j], B[2][j], \dots$). Para matrizes grandes que não cabem no cache L1/L2, cada acesso a $B[k][j]$ provoca um *cache miss*, forçando o processador a esperar centenas de ciclos pela DRAM.

5.2 A Solução por Blocagem (*Tiling*)

A blocagem particiona as matrizes em submatrizes de dimensão $B \times B$ (onde B é o tamanho do bloco ou *tile*). O tamanho do bloco é calibrado para que três sub-blocos (de A , B e C) caibam simultaneamente

no cache L1d ou L2:

$$3 \times (B \times B \times \text{sizeof}(\text{double})) \leq \text{Capacidade da Cache}$$

```
/* Multiplicação de Matrizes com Blocagem (Tiling) */
for (int ii = 0; ii < N; ii += B) {
    for (int jj = 0; jj < N; jj += B) {
        for (int kk = 0; kk < N; kk += B) {
            /* Multiplicação do bloco B x B */
            for (int i = ii; i < ii + B && i < N; i++) {
                for (int k = kk; k < kk + B && k < N; k++) {
                    double r = A[i * N + k];
                    for (int j = jj; j < jj + B && j < N; j++) {
                        C[i * N + j] += r * B[k * N + j];
                    }
                }
            }
        }
    }
}
```

Dica de Alto Desempenho: Observe que, no laço interno do bloco, trocar a ordem dos laços para i, k, j permite que o acesso mais interno a $B[k * N + j]$ e a $C[i * N + j]$ seja estritamente contíguo por linha (*Row-Major*), potencializando o vetorizador SIMD (AVX/AVX2) do compilador!

5.3 Análise Comparativa e Profiling com Valgrind / Cachegrind

Você deve implementar e compilar:

1. `matmul_padrao`: versão canônica sem blocagem.
2. `matmul_bloco`: versão otimizada com blocagem.

Compile ambas as versões em dois modos de otimização:

- Sem otimizações: `gcc -O0 -Wall programa.c -o programa_00`
- Com otimização máxima: `gcc -O3 -Wall programa.c -o programa_03`

5.3.1 Execução com o Cachegrind

Para analisar o comportamento do cache, execute as aplicações sob o utilitário `valgrind` com a ferramenta `cachegrind`:

```
$ valgrind --tool=cachegrind ./matmul_padrao_03 512
$ valgrind --tool=cachegrind ./matmul_bloco_03 512 64
```

5.3.2 Exemplo Real de Saída do Cachegrind

Abaixo são exibidos exemplos representativos de saída obtidos ao simular a multiplicação de matrizes (512×512) nas versões canônica e blocada:

1. Saída da Versão Canônica (`matmul_padrao`):

```
==24105== Cachegrind, a cache and branch-prediction profiler
==24105== Command: ./matmul_padrao_03 512
==24105==
--24105-- warning: L3 cache found, using its data for the LL simulation.
[Matmul Padrão] N: 512 | Tempo: 0.2170 s | GFLOPS: 1.24
==24105==
==24105== I    refs:      1,074,321,500
==24105== I1  misses:      1,050
==24105== LLi misses:      1,040
==24105== I1  miss rate:      0.00%
==24105== LLi miss rate:      0.00%
==24105==
```

```

==24105== D   refs:      537,162,100 (402,871,500 rd + 134,290,600 wr)
==24105== D1  misses:    33,625,400 ( 33,591,800 rd +    33,600 wr)
==24105== LLd misses:    8,410,200 ( 8,401,900 rd +    8,300 wr)
==24105== D1  miss rate:    6.3% (    8.3% +    0.0% )
==24105== LLd miss rate:    1.6% (    2.1% +    0.0% )
==24105==
==24105== LL refs:      33,626,450 ( 33,592,840 rd +    33,610 wr)
==24105== LL misses:    8,411,240 ( 8,402,940 rd +    8,300 wr)
==24105== LL miss rate:    0.5% (    0.6% +    0.0% )

```

2. Saída da Versão com Blocagem (matmul_bloco):

```

==24180== Cachegrind, a cache and branch-prediction profiler
==24180== Command: ./matmul_bloco_03 512 64
==24180==
--24180-- warning: L3 cache found, using its data for the LL simulation.
[Matmul Blocado] N: 512 | Bloco B: 64 | Tempo: 0.0261 s | GFLOPS: 10.29
==24180==
==24180== I   refs:      405,120,400
==24180== I1  misses:        1,045
==24180== LLi misses:        1,038
==24180== I1  miss rate:        0.00%
==24180== LLi miss rate:        0.00%
==24180==
==24180== D   refs:      268,580,200 (201,435,100 rd + 67,145,100 wr)
==24180== D1  misses:    2,120,400 ( 2,095,200 rd +    25,200 wr)
==24180== LLd misses:    135,100 ( 131,200 rd +    3,900 wr)
==24180== D1  miss rate:    0.8% (    1.0% +    0.0% )
==24180== LLd miss rate:    0.1% (    0.1% +    0.0% )
==24180==
==24180== LL refs:      2,121,445 ( 2,096,245 rd +    25,200 wr)
==24180== LL misses:    136,138 ( 132,238 rd +    3,900 wr)
==24180== LL miss rate:    0.0% (    0.0% +    0.0% )

```

5.3.3 Guia Didático: Onde Olhar e Como Interpretar o Relatório

Ao redigir seu relatório, concentre sua análise nos seguintes campos obrigatórios:

1. Instruções (I refs, I1 misses, LLi misses):

- Indica o total de instruções de máquina executadas pela CPU.
- Em algoritmos de álgebra linear, os laços são compactos e repetitivos. A taxa **I1 miss rate** é invariavelmente próxima de 0.00%, comprovando que o gargalo do sistema **não é o carregamento de código**, mas sim o de dados.

2. Acessos e Faltas em Cache L1 de Dados (D1 misses e D1 miss rate):

- **Onde olhar:** Observe a linha **D1 misses** e a taxa percentual **D1 miss rate** (especialmente na coluna de leitura **rd**).
- **Interpretação:** No algoritmo canônico, o salto entre linhas na matriz B faz com que a taxa de faltas em L1 salte para níveis elevados (6.3% a 10%, totalizando mais de 33 milhões de misses). Na versão blocada, ao restringir os dados aos sub-blocos $B \times B$, a taxa de falta em L1 cai para menos de 1.0% (0.8%), indicando que quase 100% das leituras foram satisfeitas diretamente na memória ultrarrápida L1d.

3. Faltas de Último Nível de Dados (LLd misses e LLd miss rate) - O PONTO MAIS CRÍTICO:

- **Onde olhar:** Compare a linha **LLd misses**: 8,410,200 na versão canônica com **LLd misses**: 135,100 na versão blocada.
- **Interpretação:** Cada falta no *Last Level Cache* (LLd miss) é uma solicitação que **não pôde ser atendida por nenhum cache (L1, L2 ou L3)** e exigiu uma viagem direta à memória principal DRAM. Como o acesso à DRAM custa entre 100 e 300 ciclos de clock (em comparação a 4 ciclos na L1), a redução de 8.4 milhões de faltas para apenas 135 mil faltas (redução superior a 98% **nas viagens à DRAM**) é a razão física pela qual a versão blocada executa quase 10× **mais rápido!**

4. Tabela-Resumo para o Relatório:

- Sintetize em seu relatório uma tabela cruzando:

Algoritmo $\times$ D1 miss rate (%) $\times$ L1d misses (absoluto) $\times$ Tempo de Execução (s)

6 Item 4: Adicionando Paralelismo com Pthreads

Após otimizar o uso da hierarquia de memória no nível sequencial, exploraremos o **paralelismo no nível de thread**.

6.1 Objetivos da Implementação Paralela

1. **matmul_pthreads.c**: Multiplicação paralela onde a matriz resultante C é particionada entre T threads.
 - A divisão mais natural consiste em particionar as N linhas da matriz entre as T threads em blocos contíguos de linhas: $\text{linhas_por_thread} = N/T$.
2. **matmul_pthreads_bloco.c**: Versão avançada integrando **Pthreads + Blocagem**, onde cada thread executa a multiplicação de sua faixa de linhas utilizando sub-blocos $B \times B$.
3. **Ausência de Mutexes**: Como cada thread escreve em posições mutuamente exclusivas da matriz C , o laço de multiplicação não deve conter locks, operando de forma 100% livre de contenção.

6.2 Cuidados com Falsa Partilha (*False Sharing*)

A falsa partilha ocorre quando duas threads em núcleos distintos leem ou escrevem em variáveis adjacentes que compartilham a mesma linha de cache de 64 bytes. Isso força o protocolo de coerência de cache (ex.: MESI/MOESI) a invalidar e transferir a linha repetidamente pelo barramento, degradando gravemente a escalabilidade.

- Ao particionar a matriz C em **blocos de linhas completas** (onde cada linha ocupa $N \times 8$ bytes $\gg 64$ bytes), a fronteira compartilhada entre threads resume-se a no máximo uma linha de cache no limiar do bloco, eliminando o problema de *False Sharing*.
-

7 Questões de Reflexão Teórico-Prática

As seguintes 8 questões devem ser respondidas com rigor analítico na Seção 5 do relatório:

1. **Localidade Espacial na Varredura (Item 2)**: Explique detalhadamente por que a varredura em ordem de linha (*Row-Major*) é substancialmente mais rápida que a varredura em ordem de coluna (*Column-Major*) na linguagem C. Em sua resposta, quantifique quantos elementos de 4 bytes cabem em uma linha de cache padrão de 64 bytes e como isso se traduz na taxa de *cache hits*.
2. **Diagnóstico de Faltas no Algoritmo Canônico (Item 3)**: Na multiplicação canônica com laços ordenados como i, j, k , qual das matrizes (A , B ou C) é a responsável pela maioria esmagadora de *cache misses* no laço mais interno? Justifique relacionando o padrão de índice com a representação na memória física.
3. **Princípio Matemático da Blocagem (Item 3)**: Como a subdivisão do domínio em ladrilhos (*tiles*) $B \times B$ converte acessos à DRAM lenta em acessos aos caches L1d e L2? Explique qual critério deve guiar a escolha da dimensão B em função da capacidade em kilobytes da memória cache do seu processador.

4. **Impacto das Otimizações do Compilador (-O0 vs. -O3):** Quais transformações de código o GCC aplica sob -O3 (e.g., auto-vetorização com instruções SIMD AVX2/AVX-512, desenrolamento de laços e pipelining de registradores)? O nível de otimização -O3 é capaz de transformar um algoritmo ruim com baixa localidade em um código eficiente, ou a estrutura algorítmica continua sendo o fator dominante?
 5. **Análise Forense de Métricas do Cachegrind / Perf (Item 3):** Compare as taxas de falha em L1 (D1 miss rate) e no cache de último nível (LLd miss rate) entre a versão padrão e a versão bloqueada. Explique por que uma pequena redução percentual nas faltas do LL Cache gera um impacto tão desproporcional no tempo global de execução.
 6. **Escalabilidade e Limites de Barramento em Pthreads (Item 4):** Ao variar a execução paralela de 1 para 2, 4, 8 e 16 threads, você obteve *Speedup* linear? Identifique os fatores de gargalo físico da arquitetura (como saturação da largura de banda do barramento de memória e concorrência pelo cache L3 compartilhado) que impedem a aceleração ideal ao escalar o número de núcleos.
 7. **Prevenção de Falsa Partilha (False Sharing) (Item 4):** Discuta como a estratégia de decomposição da matriz C adotada em sua implementação com Pthreads evitou a ocorrência de falsa partilha entre as threads. O que ocorreria se as threads alternassem a escrita elemento a elemento ou coluna a coluna?
 8. **Sinergia entre Blocagem e Concorrência Multithread (Item 4):** Quando a técnica de blocagem e o paralelismo com Pthreads são executados conjuntamente, os ganhos são aditivos ou sinérgicos? Explique como a redução de tráfego na DRAM proporcionada pelo cache local de cada núcleo melhora a escalabilidade geral do cluster multi-core.
-

8 Diretrizes para Elaboração do Relatório de Laboratório

O relatório formal em PDF deve demonstrar a completude do experimento prático, o rigor científico nas medições de hardware e a maturidade técnica na interpretação dos resultados de arquitetura e concorrência.

8.1 Formato e Identificação

- **Formato:** Exclusivamente em arquivo **PDF**.
- **Identificação:** Capa contendo título do laboratório, curso, disciplina, nome completo e RA dos integrantes do grupo e data de submissão.
- **Repositório:** Link funcional do repositório no GitHub contendo os códigos-fonte C (.c), Makefile e relatórios de profiling devidamente versionados.

8.2 Seções Obrigatórias do Relatório

1. **Introdução e Fundamentos Teóricos:** Síntese sobre hierarquia de memória (registradores, SRAM L1/L2/L3, DRAM), princípios de localidade espacial e temporal, linhas de cache, mecânica da blocagem (*tiling*) e paralelismo em memória compartilhada via Pthreads.
2. **Metodologia e Caracterização do Hardware:**
 - Modelo detalhado da CPU (frequência base, turbo, núcleos físicos e núcleos lógicos).
 - Topologia completa de caches: capacidades de L1d, L1i, L2 e L3, nível de associatividade (*n-way set associative*) e tamanho da linha de cache (bytes).
 - Capacidade e velocidade nominal da memória RAM.
 - Sistema Operacional (distribuição Linux e versão do kernel), versão do GCC e flags utilizadas.
 - Versão e comandos utilizados do utilitário de profiling (`valgrind --tool=cachegrind` ou `perf stat`).

3. **Evidências Experimentais (Capturas de Tela Obrigatórias):**
 - *Screenshots* do terminal comprovando a compilação limpa de todos os programas.
 - *Screenshots* da execução comparativa do **Item 2** (ordem de linha versus ordem de coluna), demonstrando a contagem de elementos pares e a discrepância temporal.
 - *Screenshots* dos testes do **Item 3** com matrizes grandes sob -O0 e -O3, incluindo as saídas analíticas do `valgrind` / `cachegrind` ou `perf` detalhando as taxas de miss.
 - *Screenshots* da execução paralela do **Item 4** variando entre 1, 2, 4, 8 e 16 threads, comprovando a integridade numérica dos resultados.
4. **Tabelas Comparativas de Desempenho e Gráficos:**
 - **Tabela 1 (Item 2):** Tempos e Razão de Desaceleração ($T_{\text{coluna}}/T_{\text{linha}}$) para matrizes de diferentes ordens ($N = 512$ a $N = 8192$).
 - **Tabela 2 (Item 3):** Tempo de execução (segundos), D1 miss rate (%) e LLd miss rate (%) comparando `matmul_padrao` e `matmul_bloco` sob -O0 e -O3 para $N \in \{512, 1024, 1536\}$.
 - **Tabela 3 (Item 4):** Escalabilidade paralela com Pthreads variando de 1 a 16 threads, reportando tempo (s), Speedup ($S_p = T_1/T_p$) e Eficiência ($E_p = S_p/p$).
 - **Gráficos:** Gráfico de *Speedup* versus Número de Threads e gráfico de barras comparando os *cache misses*.
5. **Respostas às 8 Questões de Reflexão:** Resoluções técnicas aprofundadas e fundamentadas com base nos dados experimentais colhidos.
6. **Seção de Dificuldades Técnicas e Soluções:** Relato transparente dos obstáculos encontrados (ex.: calibração do tamanho do bloco B , representação de matrizes em memória contígua, segmentação em matrizes grandes, interpretação das estatísticas de cache).
7. **Seção Obrigatória: Declaração e Análise do Uso de IA:**
 - Registro transparente quanto ao uso de ferramentas de IA generativa (ChatGPT, Claude, Gemini, Copilot, etc.).
 - Listagem das principais consultas/prompts formulados e em quais etapas foram aplicados.
 - **Análise Crítica Obrigatória:** Avaliação da precisão técnica das respostas, relato de eventuais códigos corrigidos ou alucinações conceituais e reflexão sobre a assimilação dos conceitos.
8. **Conclusão e Referências:** Considerações conclusivas e referências bibliográficas formais (Hennessy & Patterson, Bryant & O'Hallaron, manuais de arquitetura Intel/AMD e documentação do POSIX).

9 Critérios de Avaliação

O trabalho será avaliado considerando a completude e o rigor das seguintes dimensões:

1. **Completude da Implementação Prática (30%):** Implementação correta de todas as versões solicitadas: varredura linha/coluna (Item 2), multiplicação clássica e bloqueada (Item 3), e versões paralelas com Pthreads (Item 4), com conformidade paramétrica e integridade nos cálculos.
2. **Qualidade e Clareza do Código-Fonte (20%):** Código C limpo, modular, documentado e com compilação sem avisos (-Wall). Uso consistente de boas práticas de gerenciamento de memória e concorrência segura.
3. **Qualidade do Relatório e Evidências Experimentais (25%):** Relatório técnico em PDF bem diagramado, com caracterização exata do hardware, capturas de tela nítidas das execuções e do profiling de cache, além de tabelas estruturadas e gráficos de speedup.
4. **Profundidade Analítica e Questões de Reflexão (25%):** Rigor conceitual na interpretação dos fenômenos de cache, correlação matemática entre as dimensões de bloco e as faltas de cache, maturidade nas respostas às 8 questões de reflexão e honestidade na auditoria de IA.